Online Store Dashboard

Complete Student Guide

Express.js + Supabase + HTML/CSS/JavaScript

Project Overview

Is project mein humne ek complete Online Store Dashboard banaya hai jisme products ko add, edit, delete aur view kar sakte hain.
Pehle local server (Express) use kiya, phir Supabase (cloud) par migrate kiya.

Technologies Used

Backend:	Node.js, Express.js, CORS
Database:	JSON file (local), Supabase (cloud)
Frontend:	HTML5, CSS3, JavaScript (ES6+)
API:	REST API, Supabase JavaScript Client

Contents

1 Project Structure

1.1 Folder Structure

Sab se pehle humne project ka folder structure banaya:

```
online-store-dashboard/
|-- backend/
|   |-- package.json
|   |-- package-lock.json
|   |-- products.json
|   `-- server.js
`-- frontend/
    |-- index.html
    |-- style.css
    `-- script.js
```

Listing 1: Project Folder Structure

- **backend/** - Server-side code, data storage, API routes
- **frontend/** - User interface, styling, client-side logic
- **products.json** - Sample products ka data (3 products)

1.2 Products Data

`products.json` mein 3 sample products hain:

```
[
  {
    "id": 1,
    "name": "Wireless Mouse",
    "price": 1500,
    "stock": 25,
    "category": "Electronics"
  },
  {
    "id": 2,
    "name": "Notebook",
    "price": 200,
    "stock": 100,
    "category": "Stationery"
  },
  {
    "id": 3,
    "name": "Water Bottle",
    "price": 500,
    "stock": 50,
    "category": "Accessories"
  }
]
```

Listing 2: products.json - Sample Data

2 Express Server Setup

2.1 Installation

Backend folder mein Express install karne ke liye:

```
cd backend
npm init -y
npm install express
npm install cors
```

Listing 3: Express Installation

2.2 Server Code

`server.js` - Basic Express server:

```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// products.json file ka path
const productsFile = path.join(__dirname, 'products.json');

// Helper functions
function readProducts() {
  const data = fs.readFileSync(productsFile, 'utf8');
  return JSON.parse(data);
}

function writeProducts(products) {
  fs.writeFileSync(productsFile, JSON.stringify(products, null, 2));
}

// Root route
app.get('/', (req, res) => {
  res.send('Welcome to Online Store API!');
});

app.listen(3000, () => {
  console.log('Server chal raha hai: http://localhost:3000');
});
```

Listing 4: server.js - Express Server

2.3 Server Chalana

```
node server.js
```

Listing 5: Server Start

Output: `Server chal raha hai: http://localhost:3000`

3 CRUD API (Backend)

3.1 REST API Routes

CRUD ka matlab hai: **C**reate, **R**ead, **U**pdate, **D**elete.

Table 1: CRUD API Routes

Method	Route	Kaam	Status
GET	/api/products	Saare products	200
GET	/api/products/:id	Ek product	200/404
POST	/api/products	Naya product add	201
PUT	/api/products/:id	Product update	200/404
DELETE	/api/products/:id	Product delete	200/404

3.2 CRUD Routes Code

```
// GET /api/products -> saare products return karta hai (200)
app.get('/api/products', (req, res) => {
  const products = readProducts();
  res.status(200).json(products);
});
```

Listing 6: GET - Saare Products

```
// GET /api/products/:id -> ek product return karta hai
app.get('/api/products/:id', (req, res) => {
  const products = readProducts();
  const id = parseInt(req.params.id);
  const product = products.find(p => p.id === id);

  if (!product) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }

  res.status(200).json(product);
});
```

Listing 7: GET - Ek Product by ID

```
// POST /api/products -> naya product add karta hai (201)
app.post('/api/products', (req, res) => {
  const products = readProducts();
  const { name, price, stock, category } = req.body;

  const newId = products.length > 0
    ? Math.max(...products.map(p => p.id)) + 1
```

```
    : 1;

  const newProduct = { id: newId, name, price, stock, category };

  products.push(newProduct);
  writeProducts(products);

  res.status(201).json(newProduct);
});
```

Listing 8: POST - Naya Product Add

```
// PUT /api/products/:id -> product update karta hai
app.put('/api/products/:id', (req, res) => {
  const products = readProducts();
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }

  const { name, price, stock, category } = req.body;

  if (name !== undefined) products[index].name = name;
  if (price !== undefined) products[index].price = price;
  if (stock !== undefined) products[index].stock = stock;
  if (category !== undefined) products[index].category = category;

  writeProducts(products);
  res.status(200).json(products[index]);
});
```

Listing 9: PUT - Product Update

```
// DELETE /api/products/:id -> product delete karta hai
app.delete('/api/products/:id', (req, res) => {
  const products = readProducts();
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }

  const deletedProduct = products.splice(index, 1)[0];
  writeProducts(products);

  res.status(200).json({ message: 'Product delete ho gaya',
   deletedProduct });
});
```

Listing 10: DELETE - Product Delete

4 Frontend Design (HTML + CSS)

4.1 HTML Structure

`index.html` - Dashboard ka main structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Store Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Toast Notifications Container -->
  <div id="toastContainer" class="toast-container"></div>

  <!-- Header -->
  <header class="header">
    <h1>Online Store Dashboard</h1>
  </header>

  <!-- Main Container -->
  <main class="container">
    <!-- Toolbar -->
    <div class="toolbar">
      <button id="addProductBtn" class="btn btn-green">+ Add Product</
   button>
      <input type="text" id="searchInput" class="search-input"
   placeholder="Search products...">
    </div>

    <!-- Products Table -->
    <div class="table-wrapper">
      <table class="products-table" id="productsTable">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
            <th>Stock</th>
            <th>Category</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody id="productBody">
          <!-- Products yahan JavaScript se add honge -->
        </tbody>
      </table>
    </div>
  </main>

  <!-- Modal (Add/Edit Product) -->
  <div id="productModal" class="modal-overlay hidden">
    <div class="modal">
      <h2 id="modalTitle">Add Product</h2>
      <form id="productForm">
        <div class="form-group">
          <label for="productName">Name</label>
          <input type="text" id="productName" name="name" required>
```

```
        </div>
        <div class="form-group">
          <label for="productPrice">Price</label>
          <input type="number" id="productPrice" name="price" required>
        </div>
        <div class="form-group">
          <label for="productStock">Stock</label>
          <input type="number" id="productStock" name="stock" required>
        </div>
        <div class="form-group">
          <label for="productCategory">Category</label>
          <input type="text" id="productCategory" name="category"
    required>
        </div>
        <div class="modal-actions">
          <button type="submit" class="btn btn-green">Save</button>
          <button type="button" id="cancelBtn" class="btn btn-gray">
    Cancel</button>
        </div>
      </form>
    </div>
  </div>

  <!-- Supabase CDN -->
  <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></
    script>
  <script src="script.js"></script>
</body>
</html>
```

Listing 11: index.html - Main Structure

4.2 CSS Styling

`style.css` - Modern clean design:

- **Colors:** Light gray background (#f0f2f5), white cards
- **Buttons:** Green (Add), Blue (Edit), Red (Delete)
- **Table:** Dark header, hover effect
- **Modal:** Dark overlay + centered white form
- **Responsive:** Mobile par toolbar stack, table scroll

```
/* Toast Notifications */
.toast-container {
  position: fixed;
  top: 20px;
  right: 20px;
  z-index: 2000;
}

.toast-success { background-color: #27ae60; }
.toast-error { background-color: #e74c3c; }
```

```css

/* Buttons */
.btn-green { background-color: #27ae60; color: #ffffff; }
.btn-edit { background-color: #3498db; color: #ffffff; }
.btn-delete { background-color: #e74c3c; color: #ffffff; }

/* Table */
.products-table thead { background-color: #2c3e50; color: #ffffff; }
.products-table tbody tr:hover { background-color: #f5f7fa; }

/* Modal */
.modal-overlay {
  position: fixed;
  background-color: rgba(0, 0, 0, 0.6);
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Responsive */
@media (max-width: 768px) {
  .toolbar { flex-direction: column; }
  .btn, .btn-edit, .btn-delete { min-height: 40px; }
}
```

Listing 12: style.css - Key Styles

5 Frontend JavaScript (CRUD Logic)

5.1 Supabase Client Initialization

`script.js` - Supabase client setup:

```javascript
// ===== Supabase Client Initialization =====

const supabaseUrl = 'https://dntjgvacurwpcyavwdkc.supabase.co';
const supabaseKey = 'sb_publishable_-_6VQDTsincVrv5FwvCZow_g1F7dFHA';
const supabase = supabase.createClient(supabaseUrl, supabaseKey);

// DOMContentLoaded par loadProducts() call karo
document.addEventListener('DOMContentLoaded', () => {
  loadProducts();
});
```

Listing 13: Supabase Client Setup

5.2 Toast Notifications

```javascript
// showToast(): success/error message top-right me dikhata hai
function showToast(message, type = 'success') {
  const container = document.getElementById('toastContainer');

  const toast = document.createElement('div');
  toast.className = `toast toast-${type}`;
  toast.textContent = message;
```

```
  container.appendChild(toast);

  // 3 second baad auto-remove
  setTimeout(() => {
    toast.classList.add('hide');
    setTimeout(() => {
      toast.remove();
    }, 300);
  }, 3000);
}
```

Listing 14: Toast Notification Function

5.3 Read Products (GET)

```
// loadProducts(): Supabase se products fetch karta hai
async function loadProducts() {
  const tbody = document.getElementById('productBody');

  // Loading state dikhao
  tbody.innerHTML = '<tr><td colspan="6" style="text-align:center; color
    :#888;">Loading...</td></tr>';

  try {
    const { data: products, error } = await supabase
      .from('products')
      .select('*');

    if (error) throw error;

    renderProducts(products);
  } catch (error) {
    console.error('Products load karne me error:', error);
    tbody.innerHTML = '<tr><td colspan="6" style="text-align:center;
    color:#e74c3c;">Products load karne me error aaya. Server check
    karein.</td></tr>';
    showToast('Error: Server se connect nahi ho paya', 'error');
  }
}
```

Listing 15: loadProducts() - Supabase Select

5.4 Render Products

```
// renderProducts(): products ko table rows me render karta hai
function renderProducts(products) {
  const tbody = document.getElementById('productBody');

  // Agar products khali hain to message dikhao
  if (products.length === 0) {
    tbody.innerHTML = '<tr><td colspan="6" style="text-align:center;
    color:#888;">Koi product nahi hai</td></tr>';
    return;
  }
```

```
10
11    // Har product ki ek table row banayein
12    const rows = products.map(product => `
13      <tr>
14        <td>${product.id}</td>
15        <td>${product.name}</td>
16        <td>Rs. ${product.price}</td>
17        <td>${product.stock}</td>
18        <td>${product.category}</td>
19        <td>
20          <button class="btn-edit" data-id="${product.id}"
21                  data-name="${product.name}" data-price="${product.price
      }"
22                  data-stock="${product.stock}" data-category="${product.
      category}">Edit</button>
23          <button class="btn-delete" data-id="${product.id}">Delete</
      button>
24        </td>
25      </tr>
26    `).join('');
27
28    tbody.innerHTML = rows;
29  }
```

Listing 16: renderProducts() - Table Rows

5.5 Add Product (POST)

```
1  document.getElementById('productForm').addEventListener('submit', async
       (e) => {
2    e.preventDefault();
3
4    const name = document.getElementById('productName').value.trim();
5    const price = parseFloat(document.getElementById('productPrice').value
       );
6    const stock = parseInt(document.getElementById('productStock').value);
7    const category = document.getElementById('productCategory').value.trim
       ();
8
9    // Form validation
10   if (!name || !category) {
11     showToast('Error: Name aur Category zaroori hain!', 'error');
12     return;
13   }
14
15   if (isNaN(price) || price < 0) {
16     showToast('Error: Price negative nahi ho sakta!', 'error');
17     return;
18   }
19
20   if (isNaN(stock) || stock < 0) {
21     showToast('Error: Stock negative nahi ho sakta!', 'error');
22     return;
23   }
24
25   const productData = { name, price, stock, category };
26
```

```
  try {
    if (editingId !== null) {
      // Edit mode: Supabase update karo
      const { error } = await supabase
        .from('products')
        .update(productData)
        .eq('id', editingId);

      if (error) {
        showToast(`Error: ${error.message || 'Update me error aaya'}`, '
    error');
        return;
      }
    } else {
      // Add mode: Supabase insert karo
      const { error } = await supabase
        .from('products')
        .insert([productData]);

      if (error) {
        showToast(`Error: ${error.message || 'Add karne me error aaya'}`
    , 'error');
        return;
      }
    }

    // Success par: modal band, loadProducts() call
    document.getElementById('productModal').classList.add('hidden');
    document.getElementById('productForm').reset();
    editingId = null;
    loadProducts();
    showToast('Product saved ho gaya!', 'success');
  } catch (error) {
    console.error('Error:', error);
    showToast('Error: Server se connect nahi ho paya', 'error');
  }
});
```

Listing 17: Form Submit - Add/Edit

5.6 Delete Product (DELETE)

```
// deleteProduct(): Supabase se product delete karta hai
async function deleteProduct(id) {
  try {
    const { error } = await supabase
      .from('products')
      .delete()
      .eq('id', id);

    if (error) {
      showToast(`Error: ${error.message || 'Delete me error aaya'}`, '
    error');
      return;
    }

    // Delete ke baad loadProducts() call karo
```

```
    loadProducts();
    showToast('Product delete ho gaya!', 'success');
  } catch (error) {
    console.error('Error:', error);
    showToast('Error: Server se connect nahi ho paya', 'error');
  }
}
```

Listing 18: deleteProduct() - Supabase Delete

6 Supabase Migration

6.1 Fetch vs Supabase Comparison

Table 2: Fetch API vs Supabase Methods

Operation	Fetch (Local)	Supabase
Read	`fetch('/api/products')`	`supabase.from('products').s`
Create	`fetch('/api/products', {method:'POST'})`	`supabase.from('products').i`
Update	`fetch('/api/products/:id', {method:'PUT'})`	`supabase.from('products').u`
Delete	`fetch('/api/products/:id', {method:'DELETE'})`	`supabase.from('products').d`

6.2 Supabase Setup Steps

1. Supabase account banayein
2. Naya project create karein
3. `products` table banayein (id, name, price, stock, category)
4. Project URL aur anon key copy karein
5. CDN script HTML mein add karein
6. Client initialize karein

7 Testing

7.1 API Testing (Thunder Client / Postman)

Table 3: API Test Results

Method	URL	Body	Result
GET	/api/products	-	200 - Saare products
GET	/api/products/1	-	200 - Ek product
GET	/api/products/999	-	404 - Product nahi mila
POST	/api/products	{name, price, stock, category}	201 - Naya product
PUT	/api/products/4	{price: 2800}	200 - Update hua
DELETE	/api/products/4	-	200 - Delete hua

7.2 Browser Testing

1. Server chalayein: `node server.js`
2. Browser mein `index.html` kholen
3. Products table mein dikhne chahiye
4. Add/Edit/Delete test karein
5. Mobile view (DevTools) mein responsive check karein

8 Seekhne Ke Liye Key Concepts

1. **Project Structure** - Backend/frontend alag rakhna
2. **REST API** - CRUD operations ka concept
3. **Express.js** - Node.js server banana
4. **Fetch API** - Frontend se backend data lena
5. **CORS** - Cross-origin requests
6. **DOM Manipulation** - Table rows, modal, event listeners
7. **Async/Await** - Asynchronous JavaScript
8. **Form Validation** - User input check karna
9. **Responsive Design** - Mobile-friendly UI
10. **Toast Notifications** - User feedback
11. **Supabase** - Cloud database (Backend-as-a-Service)
12. **Migration** - Local se cloud par shift karna

9 Next Steps / Aage Kya Kar Sakte Hain

- Search functionality implement karna
- Supabase authentication add karna
- Product images add karna
- Sorting/filtering features
- Deploy karna (Netlify/Vercel for frontend)
- Pagination add karna
- Real-time updates (Supabase Realtime)

A Complete File Listings

A.1 server.js (Complete)

```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

const productsFile = path.join(__dirname, 'products.json');

function readProducts() {
  const data = fs.readFileSync(productsFile, 'utf8');
  return JSON.parse(data);
}

function writeProducts(products) {
  fs.writeFileSync(productsFile, JSON.stringify(products, null, 2));
}

// Root route
app.get('/', (req, res) => {
  res.send('Welcome to Online Store API!');
});

// GET /api/products
app.get('/api/products', (req, res) => {
  const products = readProducts();
  res.status(200).json(products);
});

// GET /api/products/:id
app.get('/api/products/:id', (req, res) => {
```

```
  const products = readProducts();
  const id = parseInt(req.params.id);
  const product = products.find(p => p.id === id);

  if (!product) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }

  res.status(200).json(product);
});

// POST /api/products
app.post('/api/products', (req, res) => {
  const products = readProducts();
  const { name, price, stock, category } = req.body;

  const newId = products.length > 0
    ? Math.max(...products.map(p => p.id)) + 1
    : 1;

  const newProduct = { id: newId, name, price, stock, category };

  products.push(newProduct);
  writeProducts(products);

  res.status(201).json(newProduct);
});

// PUT /api/products/:id
app.put('/api/products/:id', (req, res) => {
  const products = readProducts();
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }

  const { name, price, stock, category } = req.body;

  if (name !== undefined) products[index].name = name;
  if (price !== undefined) products[index].price = price;
  if (stock !== undefined) products[index].stock = stock;
  if (category !== undefined) products[index].category = category;

  writeProducts(products);
  res.status(200).json(products[index]);
});

// DELETE /api/products/:id
app.delete('/api/products/:id', (req, res) => {
  const products = readProducts();
  const id = parseInt(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1) {
    return res.status(404).json({ message: 'Product nahi mila' });
  }
```

```
94
95    const deletedProduct = products.splice(index, 1)[0];
96    writeProducts(products);
97
98    res.status(200).json({ message: 'Product delete ho gaya',
       deletedProduct });
99  });
100
101 app.listen(3000, () => {
102   console.log('Server chal raha hai: http://localhost:3000');
103 });
```

Listing 19: server.js - Full Code

A.2 script.js (Complete)

```
1  // ===== Supabase Client Initialization =====
2
3  const supabaseUrl = 'https://dntjgvacurwpcyavwdkc.supabase.co';
4  const supabaseKey = 'sb_publishable_-_6VQDTsincVrv5FwvCZow_g1F7dFHA';
5  const supabase = supabase.createClient(supabaseUrl, supabaseKey);
6
7  // DOMContentLoaded par loadProducts() call karo
8  document.addEventListener('DOMContentLoaded', () => {
9    loadProducts();
10 });
11
12 // ===== Toast Notifications =====
13
14 function showToast(message, type = 'success') {
15   const container = document.getElementById('toastContainer');
16
17   const toast = document.createElement('div');
18   toast.className = `toast toast-${type}`;
19   toast.textContent = message;
20
21   container.appendChild(toast);
22
23   setTimeout(() => {
24     toast.classList.add('hide');
25     setTimeout(() => {
26       toast.remove();
27     }, 300);
28   }, 3000);
29 }
30
31 // ===== Read Products =====
32
33 async function loadProducts() {
34   const tbody = document.getElementById('productBody');
35
36   tbody.innerHTML = '<tr><td colspan="6" style="text-align:center; color
      :#888;">Loading...</td></tr>';
37
38   try {
39     const { data: products, error } = await supabase
40       .from('products')
```

```
      .select('*');

    if (error) throw error;

    renderProducts(products);
  } catch (error) {
    console.error('Products load karne me error:', error);
    tbody.innerHTML = '<tr><td colspan="6" style="text-align:center;color:#e74c3c;">Products load karne me error aaya. Server check karein.</td></tr>';
    showToast('Error: Server se connect nahi ho paya', 'error');
  }
}

// ===== Render Products =====

function renderProducts(products) {
  const tbody = document.getElementById('productBody');

  if (products.length === 0) {
    tbody.innerHTML = '<tr><td colspan="6" style="text-align:center;color:#888;">Koi product nahi hai</td></tr>';
    return;
  }

  const rows = products.map(product => `
    <tr>
      <td>${product.id}</td>
      <td>${product.name}</td>
      <td>Rs. ${product.price}</td>
      <td>${product.stock}</td>
      <td>${product.category}</td>
      <td>
        <button class="btn-edit" data-id="${product.id}"
                data-name="${product.name}" data-price="${product.price}"
                data-stock="${product.stock}" data-category="${product.category}">Edit</button>
        <button class="btn-delete" data-id="${product.id}">Delete</button>
      </td>
    </tr>
  `).join('');

  tbody.innerHTML = rows;
}

// ===== Modal Open/Close =====

document.getElementById('addProductBtn').addEventListener('click', () => {
  editingId = null;
  document.getElementById('modalTitle').textContent = 'Add Product';
  document.getElementById('productForm').reset();
  document.getElementById('productModal').classList.remove('hidden');
});

document.getElementById('cancelBtn').addEventListener('click', () => {
```

```
  document.getElementById('productModal').classList.add('hidden');
});

document.getElementById('productModal').addEventListener('click', (e) => {
  if (e.target === document.getElementById('productModal')) {
    document.getElementById('productModal').classList.add('hidden');
  }
});

// ===== Form Submit (Add/Edit) =====

let editingId = null;

document.getElementById('productForm').addEventListener('submit', async (e) => {
  e.preventDefault();

  const name = document.getElementById('productName').value.trim();
  const price = parseFloat(document.getElementById('productPrice').value);
  const stock = parseInt(document.getElementById('productStock').value);
  const category = document.getElementById('productCategory').value.trim();

  if (!name || !category) {
    showToast('Error: Name aur Category zaroori hain!', 'error');
    return;
  }

  if (isNaN(price) || price < 0) {
    showToast('Error: Price negative nahi ho sakta!', 'error');
    return;
  }

  if (isNaN(stock) || stock < 0) {
    showToast('Error: Stock negative nahi ho sakta!', 'error');
    return;
  }

  const productData = { name, price, stock, category };

  try {
    if (editingId !== null) {
      const { error } = await supabase
        .from('products')
        .update(productData)
        .eq('id', editingId);

      if (error) {
        showToast(`Error: ${error.message || 'Update me error aaya'}`, 'error');
        return;
      }
    } else {
      const { error } = await supabase
        .from('products')
        .insert([productData]);
```

```
145
146       if (error) {
147         showToast(`Error: ${error.message || 'Add karne me error aaya'}`
    , 'error');
148         return;
149       }
150     }
151
152     document.getElementById('productModal').classList.add('hidden');
153     document.getElementById('productForm').reset();
154     editingId = null;
155     loadProducts();
156     showToast('Product saved ho gaya!', 'success');
157   } catch (error) {
158     console.error('Error:', error);
159     showToast('Error: Server se connect nahi ho paya', 'error');
160   }
161 });
162
163 // ===== Edit & Delete Events =====
164
165 document.getElementById('productBody').addEventListener('click', (e) =>
    {
166   if (e.target.classList.contains('btn-edit')) {
167     const id = e.target.dataset.id;
168     const name = e.target.dataset.name;
169     const price = e.target.dataset.price;
170     const stock = e.target.dataset.stock;
171     const category = e.target.dataset.category;
172
173     editingId = parseInt(id);
174     document.getElementById('modalTitle').textContent = 'Edit Product';
175
176     document.getElementById('productName').value = name;
177     document.getElementById('productPrice').value = price;
178     document.getElementById('productStock').value = stock;
179     document.getElementById('productCategory').value = category;
180
181     document.getElementById('productModal').classList.remove('hidden');
182   }
183
184   if (e.target.classList.contains('btn-delete')) {
185     const id = e.target.dataset.id;
186
187     if (confirm('Kya aap yeh product delete karna chahte hain?')) {
188       deleteProduct(id);
189     }
190   }
191 });
192
193 // ===== Delete Product =====
194
195 async function deleteProduct(id) {
196   try {
197     const { error } = await supabase
198       .from('products')
199       .delete()
200       .eq('id', id);
```

```
201
202     if (error) {
203       showToast(`Error: ${error.message || 'Delete me error aaya'}`, '
    error');
204       return;
205     }
206
207     loadProducts();
208     showToast('Product delete ho gaya!', 'success');
209   } catch (error) {
210     console.error('Error:', error);
211     showToast('Error: Server se connect nahi ho paya', 'error');
212   }
213 }
```

Listing 20: script.js - Full Code